WORKING AGREEMENT OF TEAM WORK
ACUERDO INTERNO DEL EQUIPO

Acuerdos Operativos, Gobierno y Estándares de Ingeniería de Software para la Ejecución del Plan de Desarrollo de Software con Metodología Híbrida RUP + SCRUM

Proyecto	Sistema de Información para la Inclusión Socioeconómica y Cultural de Personas con Discapacidad
Ámbito de Aplicación	Región Tacna (Provincias de Tacna, Tarata, Jorge Basadre y Candarave)
Marco Metodológico	Metodología Híbrida: RUP (Gobierno, Arquitectura, Trazabilidad) + SCRUM (Agilidad, Sprints)
Plazo Base de Ejecución	28 días calendario (4 Sprints de 1 semana) + 12 meses de Soporte Post-Implementación
Versión del Documento	Versión 1.0 — Documento de Gobierno Formal
Estado de Aprobación	Aprobado por Consenso del Equipo de Desarrollo y Product Owner

Índice del Documento

Nota: La tabla de contenidos se actualizará automáticamente al abrir el documento en Microsoft Word o Google Docs.

1. Resumen Ejecutivo del Acuerdo de Trabajo

El presente Working Agreement constituye la norma rectora de convivencia, organización técnica, procesos de ingeniería y compromisos de calidad del equipo de desarrollo de software. Este documento integra las mejores prácticas prescritas en el Rational Unified Process (RUP) para el gobierno arquitectónico, modelado UML y trazabilidad formal, conjuntamente con el marco ágil Scrum para la gestión dinámica del Product Backlog, la ejecución de Sprints semanales y la entrega continua de incrementos funcionales.

Tabla de Referencia Rápida de Acuerdos Operativos

Dimensión / Ámbitos	Acuerdo Operativo Estándar	Herramienta / Regla Principal
Canal Oficial de Comunicación	Mensajería general e incidencias técnicas. Prohibido aprobar cambios en chats informales.	Discord / Teams / WhatsApp Oficial
Sincronización Diaria (Daily)	Reunión obligatoria de 15 min. Máximo 3 min por integrante. Mantenimiento del foco.	Google Meet / Teams (Mismo enlace siempre)
Gestión de Requisitos & Trazabilidad	Casos de Uso RUP mapeados a User Stories en Product Backlog. Matriz de trazabilidad activa.	GitHub Projects / Google Drive Documental
Repositorio de Código & Ramas	Control de versiones Git centralizado. Ramas: main, develop, feature/*, fix/*.	GitHub Repositorio Oficial
Estrategia de Commits	Commits pequeños y descriptivos siguiendo convención semántica.	Tipo: feat, fix, docs, test, refactor, chore
Integración & Code Review	Pull Requests obligatorios. Mínimo 1 aprobación técnica antes de merge a develop.	Peer Code Review en GitHub PRs
Filtros de Calidad (DoR / DoD)	Definition of Ready obligatoria para entrar al Sprint; Definition of Done para cerrar tareas.	Checklists formales aprobadas por QA y PO
Escalación de Impedimentos	Notificación inmediata al Scrum Master si una traba supera las 2 horas sin avance.	Canal de Impedimentos + Daily Scrum
Ambiente de Despliegue	Despliegue continuo de incrementos en VPS institucional del Gobierno Regional de Tacna.	VPS Linux (Apache/Nginx, Vue.js, CodeIgniter, MySQL)

2. Introducción y Objetivos del Working Agreement

El desarrollo de sistemas de información complejos en el sector público requiere combinar la rigurosidad metodológica con la agilidad en la entrega de valor. El presente Working Agreement establece los lineamientos operativos, las normas de conducta, la distribución de responsabilidades y los mecanismos de ingeniería que guiarán al equipo de trabajo durante la construcción e implementación del Sistema de Información para las personas con discapacidad en las cuatro provincias del departamento de Tacna.

2.1 Objetivo General

Establecer un marco normativo e institucional interno que regule la interacción, comunicación, toma de decisiones, flujo de desarrollo e ingeniería de calidad del equipo de proyecto, garantizando el cumplimiento estricto del plazo de 28 días calendario mediante la integración disciplinada de las fases de RUP y la ejecución iterativa de Sprints en Scrum.

2.2 Objetivos Específicos

- **Canales y Gobierno:** Definir los canales oficiales de comunicación e intercambio de artefactos, eliminando la dispersión de acuerdos informales.
- **Roles e Integración:** Formalizar la estructura organizativa mediante una Matriz RACI que alinee los roles tradicionales de RUP (Analistas, Arquitectos, QA, DBA) con la dinámica de Scrum (Product Owner, Scrum Master, Developers).
- **Aseguramiento de Calidad:** Establecer criterios unificados para la Definition of Ready (DoR) y Definition of Done (DoD), asegurando que ningún componente sea considerado finalizado sin la debida verificación de calidad y trazabilidad.
- **Estándares de Código:** Normalizar el flujo de trabajo en el control de versiones (Git), definiendo reglas para la creación de ramas, convención semántica de commits y revisiones obligatorias de código (Code Review).
- **Gestión de Conflictos:** Proveer un protocolo formal para la detección, reporte y resolución de impedimentos, riesgos y desacuerdos técnicos o funcionales.

3. Fundamentos de la Metodología Híbrida RUP + SCRUM

La adopción de un enfoque híbrido responde a la necesidad contractual de cumplir con la entrega de documentación formal, arquitectura robusta y modelado UML (propios de RUP) en un horizonte de tiempo sumamente acotado de 28 días calendario, donde la capacidad de respuesta al cambio y las demostraciones semanales al área usuaria (propias de Scrum) resultan indispensables.

3.1 Integración de Fases RUP y Sprints de Scrum

Para evitar la dualidad operativa o la fricción entre métodos, RUP actúa como el marco contenedor de gobierno de ingeniería y arquitectura, mientras que Scrum opera como el motor de ejecución diaria por medio de Sprints de 1 semana:

- **Fase de Inicio RUP:** Alineada con el Sprint 0 (Días 1 a 3). Se realiza la alineación inicial, la especificación de alcance, la evaluación de riesgos, la preparación del entorno de desarrollo y la construcción del Product Backlog inicial.

- **Fase de Elaboración RUP:** Alineada con el Sprint 1 (Días 4 a 10). Se profundiza el levantamiento de procesos de negocio, la especificación de casos de uso RUP primarios, el diseño de la arquitectura base, el modelado de datos MySQL y la validación del prototipo con el área usuaria.
- **Fase de Construcción RUP:** Alineada con los Sprints 2 y 3 (Días 11 a 24). Se ejecuta el desarrollo incremental de los módulos funcionales del sistema (Registro de Administrados/Funcionarios, Fichas Técnicas, Seguimiento, Estadísticas y Reportes) bajo disciplina de construcción, pruebas y revisiones de código.
- **Fase de Transición RUP:** Alineada con el Sprint 4 y Release Final (Días 25 a 28). Se realizan las pruebas de aceptación de usuario (UAT), el despliegue en el VPS institucional, la entrega del código fuente versionado, la capacitación de usuarios OREDIS/OMAPED y la firma del acta de conformidad.

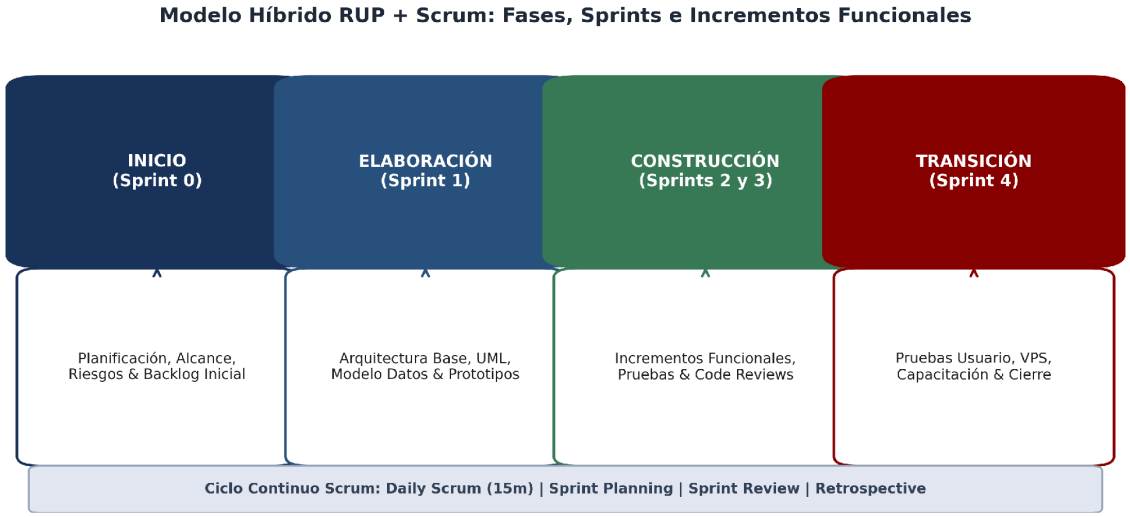


Figura 1: Mapeo de Fases RUP e Iteraciones/Sprints de Scrum en el Plan de Trabajo de 28 Días.

4. Principios Culturales y Valores del Equipo

El éxito del proyecto no depende únicamente de las herramientas de software empleadas, sino de la cohesión humana y la madurez profesional del equipo. Inspirados en los principios del Manifiesto Ágil, la Guía de Scrum 2020 y las prácticas de ingeniería de software de alto rendimiento (Google Software Engineering / Pressman), el equipo asume los siguientes compromisos:

- **Humildad Profesional:** Reconocer que el código y los artefactos son propiedad colectiva del equipo. Nadie adoptará una postura defensiva ante observaciones o críticas técnicas constructivas durante las revisiones.
- **Respeto Mutuo:** Tratar a todos los integrantes con consideración y empatía, valorando la especialidad técnica de cada rol (Analistas, Desarrolladores, QA, DBA, PO).
- **Confianza y Autonomía:** Delegar autoridad y asumir la responsabilidad individual sobre los compromisos adquiridos en el Sprint Planning. Fomentar una cultura Blameless (sin culpabilización) orientada al análisis de causa raíz.
- **Transparencia Radical:** Reflejar el estado real del proyecto en el tablero de gestión. Ocultar retrasos o impedimentos es considerado una falta grave contra el equipo.

- **Foco y Disciplina:** Mantener el enfoque principal en el cumplimiento del Sprint Goal. Se evitará la incorporación de características no solicitadas (Gold Plating) que pongan en riesgo el cronograma.

5. Estructura de Organización, Roles y Matriz RACI

Para asegurar una gobernanza clara sin burocracia excesiva, se establece una estructura organizativa que mapea explícitamente las responsabilidades entre el equipo cliente/área usuaria y el equipo técnico de desarrollo.

Actividad Principal del Proyecto	Jefe Proy / PM	Product Owner	Scrum Master	Líder Técnico	Analista / QA	Developers
Definición de Alcance & Prioridades	**A**	**R**	C	C	C	I
Levantamiento de Requisitos & Casos Uso	C	**A**	C	C	**R**	C
Diseño de Arquitectura Base & Modelo Datos	**A**	I	C	**R**	C	C
Planificación del Sprint (Sprint Planning)	I	**A**	F / C	C	C	**R**
Desarrollo de Software (Frontend/Backend)	I	I	C	**A**	C	**R**
Ejecución de Pruebas Unitarias & QA	I	I	I	**A**	**R**	**R**
Code Review & Aprobación de Pull Request	I	I	I	**A**	C	**R**
Demostración de Incremento (Sprint Review)	C	**A**	F	C	**R**	**R**

Pruebas de Aceptación de Usuario (UAT)	C	**A**	I	C	**R**	I
Despliegue en VPS Institucional & Cierre	**A**	**A**	C	**R**	C	**R**

Simbología RACI: R = Responsable de la ejecución; A = Aprueba / Accountable final; C = Consultado; I = Informado; F = Facilitador.

6. Canales de Comunicación, Horarios y Disponibilidad

Para optimizar el flujo de información y reducir interrupciones no planificadas, el equipo acuerda estructurar sus interacciones bajo los siguientes canales oficiales:

- **Canal de Comunicación General (Discord / Teams / WhatsApp):** Mensajería instantánea para la coordinación del día a día, avisos rápidos y alertas de impedimentos. No se aprobarán cambios de alcance por este medio.
- **Videoconferencias (Google Meet / Teams):** Plataforma para reuniones síncronas (Daily, Planning, Review, Retrospective). Se mantendrá un único enlace persistente.
- **Repositorio Documental (Google Drive):** Repositorio centralizado para el almacenamiento de documentación técnica, Actas, Modelo de Casos de Uso y Matriz de Trazabilidad.
- **Gestión de Código y Tareas (GitHub):** Repositorio único para el código fuente, gestión de incidencias (Issues), tablero de control del backlog (Projects) y revisiones de código (Pull Requests).

Reglas de Horarios y Disponibilidad

- **Horario Laboral Preferente:** Días hábiles de Lunes a Viernes de 08:30 a 18:00 hrs. La Daily Scrum se ejecutará de forma estricta a las 09:00 hrs.
- **Puntualidad en Ceremonias:** Inicia a las 09:00 hrs puntualmente. Tolerancia máxima de 3 minutos. La ausencia injustificada afectará el indicador de compromiso del integrante.
- **Notificación de Ausencias:** Si un integrante prevé su ausencia o indisponibilidad, deberá comunicarlo con un mínimo de 12 horas de anticipación a través del canal oficial.

7. Normativa para las Ceremonias y Eventos Scrum

Las ceremonias dentro del marco híbrido están diseñadas para mantener la cadencia del proyecto, inspeccionar los avances y adaptar el plan de trabajo rápidamente.

7.1 Daily Scrum (Sincronización Diaria)

La Daily Scrum es una reunión de sincronización de 15 minutos para los Developers. Se realizará todos los días a las 09:00 hrs. Cada integrante responderá de forma concisa:

- ¿Qué actividades completé desde la última Daily para aportar al Sprint Goal?
- ¿Qué actividades ejecutaré hoy para avanzar hacia el Sprint Goal?
- ¿Qué impedimentos o bloqueos técnicos/funcionales identifico que me impiden avanzar?

Regla Antipatrón: Regla de Parking Lot: Se prohíbe debatir soluciones técnicas complejas durante la Daily. Si surge un tema técnico detallado, se anota en el 'Parking Lot' y se discute inmediatamente después con las personas involucradas.

7.2 Sprint Planning (Planificación de la Iteración)

Se ejecutará el primer día de cada Sprint con una duración máxima de 2 horas. Durante esta sesión, el Product Owner presenta los ítems priorizados del Product Backlog que cumplen con la Definition of Ready (DoR). El equipo de desarrollo selecciona los ítems según su capacidad real y define el Sprint Goal.

7.3 Sprint Review (Demostración de Incrementos)

Se realizará el último día del Sprint. El equipo presentará un incremento de software totalmente funcional y desplegado en el servidor de pruebas/VPS. No se presentarán diapositivas estáticas ni conceptos no codificados; únicamente software operativo que cumpla con la Definition of Done (DoD).

7.4 Sprint Retrospective (Mejora Continua)

Sesión interna de 45 minutos celebrada tras la Sprint Review. El equipo analiza qué funcionó bien, qué problemas surgieron y qué compromisos de mejora se incorporarán en el Working Agreement para el siguiente Sprint.

8. Gestión de Requisitos, Artefactos y Trazabilidad

La trazabilidad es el pilar central que garantiza la conformidad contractual y la calidad de la ingeniería de software. Cada necesidad del área usuaria debe ser rastreable a lo largo de todo el ciclo de vida del desarrollo.

La cadena de trazabilidad obligatoria se define de la siguiente manera:

Necesidad Institucional → Caso de Uso RUP → Elemento del Product Backlog (User Story) → Criterios de Aceptación → Tarea de Desarrollo Git → Componente de Código → Caso de Prueba QA → Evidencia de Aceptación.



Figura 2: Cadena de Trazabilidad RUP/Scrum y Puertas de Calidad (DoR y DoD).

9. Control de Versiones (Git), Commits y Code Review

Para mantener la integridad de la base de código y permitir el trabajo concurrente sin colisiones destructivas, se establecen las siguientes reglas obligatorias de control de versiones:

9.1 Estrategia de Ramas (GitFlow Adaptado)

- **main / master:** Contiene exclusivamente código estable en producción. Solo se actualiza mediante merges aprobados desde develop o hotfix.
- **develop:** Rama de integración continua. Todo el desarrollo de características se integra en esta rama.
- **feature/*:** Creadas a partir de develop para construir un módulo o historia específica. Nombre obligatorio: feature/RF-XXX-descripcion.
- **fix/*:** Ramas para corrección de defectos encontrados en QA. Nombre: fix/DEF-XXX-descripcion.

9.2 Convención Semántica de Commits

Los commits deberán ser atómicos, frecuentes y redactados bajo la siguiente estructura semántica:

- feat: incorporar registro masivo de administrados por provincia
- fix: corregir validación de DNI en módulo OMAPED
- docs: actualizar matriz de trazabilidad y especificación de casos de uso
- test: agregar casos de prueba unitarios para controladores Vue.js

9.3 Protocolo de Pull Requests (PR) y Code Review

Queda estrictamente prohibido realizar push directo a las ramas main o develop. Toda integración debe realizarse vía Pull Request (PR) cumpliendo con el siguiente procedimiento:

- **Apertura de PR:** El desarrollador abre el PR vinculando el Issue/Tarea de GitHub correspondiente.
- **Revisión Obligatoria:** Al menos UN desarrollador (Peer Reviewer) o el Líder Técnico deben revisar el código.
- **Criterios de Aprobación:** El revisor verificará que el código no rompa funcionalidades existentes, cumpla con la arquitectura, no contenga credenciales expuestas y pase las pruebas unitarias.
- **Tono Constructivo:** Las observaciones en el PR se realizarán con tono amable y profesional, enfocado en el código y no en la persona.

10. Filtros de Calidad: Definition of Ready (DoR) y Definition of Done (DoD)

Los filtros DoR y DoD constituyen las puertas de calidad contractuales que previenen el ingreso de requisitos ambiguos y la entrega de código defectuoso.

DEFINITION OF READY (DoR) (Filtro para ingresar al Sprint)	DEFINITION OF DONE (DoD) (Filtro para cerrar el Incremento)

☐ Requisito/Historia con descripción clara y origen validado. ☐ Actor o perfil de usuario identificado. ☐ Criterios de Aceptación explícitos e inambiguos. ☐ Dependencias técnicas/funcionales identificadas. ☐ Estimado por el equipo de desarrollo. ☐ Mockups o bocetos de interfaz disponibles. ☐ Aprobado por el Product Owner.	☐ Código fuertemente compilado sin advertencias. ☐ Repositorio Git actualizado mediante PR aprobado. ☐ Code Review realizado y observaciones cerradas. ☐ Pruebas unitarias y funcionales ejecutadas sin fallas. ☐ Cero defectos de prioridad Alta/Crítica abiertos. ☐ Trazabilidad en Matriz RUP/Scrum actualizada. ☐ Desplegado en entorno VPS de Pruebas/Piloto. ☐ Demostrado y aceptado por el Product Owner.

11. Gestión de Impedimentos, Riesgos y Conflictos

Cuando se identifican trabas operativas o discrepancias técnicas, el equipo seguirá un procedimiento ágil de resolución para proteger el flujo de trabajo.

11.1 Escalación de Impedimentos

Un impedimento es cualquier bloqueo que detiene el avance de una tarea por más de 2 horas. El procedimiento de escalación incluye:

- **Paso 1:** El desarrollador registra la incidencia en GitHub Issues y la notifica al Scrum Master por el canal de mensajes.
- **Paso 2:** El Scrum Master evalúa si el bloqueo es técnico interno o externo (ej. falta de aclaración funcional del PO o accesos a servidor).
- **Paso 3:** Si es externo, el SM coordina con el PO o el cliente institucional para destrabar el requerimiento en un plazo no mayor a 24 horas.

11.2 Resolución de Desacuerdos Técnicos y Funcionales

- **Fase 1 (Consenso Técnico):** Debate técnico abierto entre los involucrados aportando argumentos basados en evidencia.
- **Fase 2 (Decisión Técnica):** Si no hay acuerdo en 30 minutos, el Líder Técnico tomará la decisión arquitectónica basada en estándares de ingeniería.
- **Fase 3 (Decisión de Negocio):** Si el desacuerdo es de carácter funcional o de negocio, la decisión final corresponde exclusivamente al Product Owner.

12. Control de Cambios y Gobierno del Acuerdo

Dado que el proyecto cuenta con un horizonte cerrado de 28 días calendario, todo cambio de alcance debe ser gestionado bajo estricto control de impacto.

- **Solicitud Formal:** No se aceptarán solicitudes de cambio verbales ni por chats informales. Toda solicitud debe ser formulada por el PO mediante el formato formal de Control de Cambios.
- **Evaluación de Impacto:** El Jefe de Proyecto y el Líder Técnico evaluarán el impacto en días calendario, arquitectura y costo. Si un nuevo requisito es incorporado, un requisito de esfuerzo equivalente debe ser retirado del Product Backlog (Trueque de Alcance).
- **Evolución del Acuerdo:** El Working Agreement es un documento vivo. Se revisará al final de cada Sprint durante la Retrospectiva para realizar ajustes y mejoras continuas.

13. Compromiso y Firmas de Conformidad del Equipo

Los abajo firmantes declaran conocer, comprender y aceptar en su totalidad las cláusulas, reglas operativas y compromisos de calidad establecidos en el presente Working Agreement, comprometiéndose a velar por su estricto cumplimiento durante la ejecución del proyecto.

Nombre y Apellido	Rol en el Proyecto	Conformidad	Fecha
[Nombre del Integrante]	**Product Owner (Área Usuaria)**	CONFORME	[DD/MM/AAAA]
[Nombre del Integrante]	**Scrum Master / Facilitador**	CONFORME	[DD/MM/AAAA]
[Nombre del Integrante]	**Consultor Responsable / Líder Técnico**	CONFORME	[DD/MM/AAAA]
[Nombre del Integrante]	**Analista Funcional / Sistemas**	CONFORME	[DD/MM/AAAA]
[Nombre del Integrante]	**Developer Frontend / Backend**	CONFORME	[DD/MM/AAAA]
[Nombre del Integrante]	**Analista de Calidad (QA) / DBA**	CONFORME	[DD/MM/AAAA]